Roll # Section: Name:

Department: **Software Engineering** Program: **BS (SE)**

Assignment 1
SE – 203T Data Structures and Algorithms

Announced Date:**27-10-2026** Due Date:**05-11-2026**
Total Marks: 3 Marks
Marks Obtained: ____

Teacher Name: Ms. Zainab Zakir, Ms. Falak Saleem, Ms Sarwat Fawwad

Sr. No	Course Learning Outcomes	GAs	Blooms Taxonomy
CLO1	**Explain** fundamental data structures as well as algorithms and their complexities.	GA_2 (Knowledge for solving computing problem)	C2 (Understand)

Q1) **Inferring** the following scenarios with at least 2 data structures with reason which are suitable for it. Also write the Pseudocode for the following scenarios. The system wants to develop an Online Shopping Order Management System that stores the prices of products in an array. The rules for classifying the products are as follows:

i. Store the prices of all products in an array.
ii. Count how many products are low-priced (price below 1,000) and how many are high-priced (price 1,000 or above).
iii. Calculate the discount amount for each product priced at 5,000 or above, assuming a 10% discount. Products priced below 5,000 will receive no discount.
iv. At the end, display the total counts of low-priced, high-priced, discount-eligible, and non-eligible products.

Answer:

(a) Suitable Data Structures

Data Structure	Reason for Suitability
1. Array	Prices are stored in contiguous memory, so any price can be accessed directly by its index in **O(1)**. A single loop over the array is enough to count, classify and calculate discount in **O(n)**. Rule (i) itself requires an array.
2. ArrayList (Dynamic Array)	In an online store, products are added or removed often. An ArrayList resizes automatically and still gives O(1) index access. A second list can hold only the discount-eligible products.

3. HashMap (additional)	Stores *ProductID* → *Price* pairs, so the price of a specific product can be searched or updated in O(1) average time without scanning the whole array.

(b) Pseudocode

```
ALGORITHM ProductOrderClassification
BEGIN
    INPUT n                                     // total number of products
    DECLARE price[n], discount[n]
    SET lowCount = 0, highCount = 0
    SET eligibleCount = 0, nonEligibleCount = 0

    // Rule (i): store prices in array
    FOR i = 0 TO n-1 DO
        INPUT price[i]
    END FOR

    FOR i = 0 TO n-1 DO
        // Rule (ii): low-priced / high-priced
        IF price[i] < 1000 THEN
            lowCount = lowCount + 1
        ELSE
            highCount = highCount + 1
        END IF

        // Rule (iii): 10% discount on price >= 5000
        IF price[i] >= 5000 THEN
            discount[i] = price[i] * 0.10
            eligibleCount = eligibleCount + 1
        ELSE
            discount[i] = 0
            nonEligibleCount = nonEligibleCount + 1
        END IF
        PRINT "Price:", price[i], " Discount:", discount[i]
    END FOR

    // Rule (iv): display totals
    PRINT "Low-priced products        :", lowCount
    PRINT "High-priced products       :", highCount
    PRINT "Discount-eligible products :", eligibleCount
    PRINT "Non-eligible products      :", nonEligibleCount
END
```

Example: price = [500, 1200, 6000, 800, 10000]

Price	Category	Discount (10%)
500	Low	0
1200	High	0
6000	High	600
800	Low	0
10000	High	1000

Low-priced = **2**, High-priced = **3**, Discount-eligible = **2**, Non-eligible = **3**. Time Complexity = **O(n)**, Space Complexity = **O(n)**.

Q2) **Estimates** the time complexity of the following code

```
for (int i = 0; i < n; i++) {
  if (i % 2 == 0) {
    for (int j = 1; j < n; j *= 2) {
      for (int k = 0; k < n; k++) {
        System.out.println(k);
      }
    }
  } else {
    for (int m = 1; m < n; m *= 2) {
      System.out.println(m);
    }

    int p = 0;
    do {
      System.out.println(p);
      p++;
    }
    while (p < n);
  }
}
```

Answer:

Loop / Statement	Executes	Reason
Outer loop *i*	n	i goes 0 to n-1 (n/2 even, n/2 odd)
Even i: loop *j* (j *= 2)	$\log_2 n$	j doubles: 1, 2, 4, ... < n
inner loop *k*	n	k goes 0 to n-1
Even total (per i)	n log n	log n × n
Odd i: loop *m* (m *= 2)	$\log_2 n$	m doubles: 1, 2, 4, ... < n
do-while loop *p*	n	p goes 0 to n-1
Odd total (per i)	log n + n	sequential loops are added

Total:

$$T(n) = (n/2) \times (n \log n) + (n/2) \times (\log n + n)$$
$$T(n) = (n^2 \log n)/2 + (n \log n)/2 + n^2/2$$

Dropping constants and lower-order terms, the dominant term is $n^2 \log n$.

Time Complexity = $O(n^2 \log n)$

Q3) **Explain** the following recursive functions and also **show** the Recursion Tree.

```
public class Main {
  public static void main(String[] args) {
    int result = power(2, 4);
    System.out.println(result);
  }

  public static int power(int base, int exponent) {
    if (exponent == 0) {
      return 1
    } else {
      return base * power(base, exponent - 1);
    }
  }
}
```

Answer:

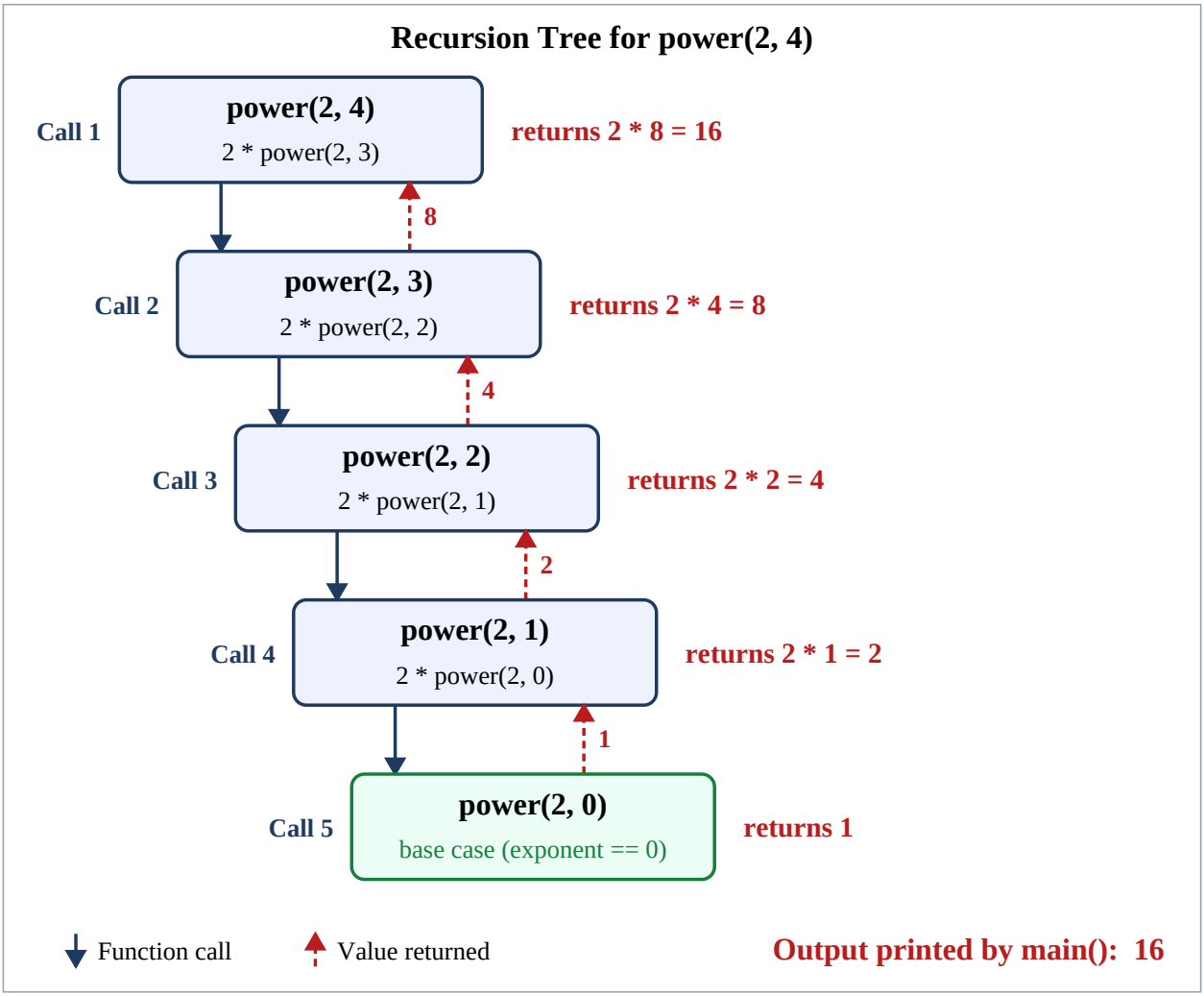
(a) Explanation

The function **power(base, exponent)** calculates $base^{exponent}$ by recursion, using the rule $base^e = base \times base^{e-1}$.

- **Base case:** when exponent == 0 the function returns 1 (any number to the power 0 is 1). This stops the recursion.
- **Recursive case:** otherwise it returns base * power(base, exponent - 1). Each call reduces the exponent by 1, so it always moves towards the base case.

- main() calls power(2, 4). The calls go down to power(2, 0), then the results are multiplied while returning back up: 1 → 2 → 4 → 8 → 16. So the program prints **16**.
- This is **linear recursion**: each call makes only one recursive call, so power(2, 4) makes 5 calls in total.
- **Time Complexity = O(n)** and **Space Complexity = O(n)** (call stack), where n = exponent.
- **Note:** the given code has a syntax error: *return 1* must be written as *return 1;*

(b) Recursion Tree



Call	Expression	Returned Value
power(2, 0)	base case	1
power(2, 1)	2 × 1	2
power(2, 2)	2 × 2	4
power(2, 3)	2 × 4	8
power(2, 4)	2 × 8	**16**

Final Output: 16